KeenPay

Production Architecture & Systems Design

Document Status: Approved for Implementation

Author: Lead Systems Architect

Grow | Sell | Protect

FastAPI | LangGraph | PostgreSQL | Redis | Next.js | Razorpay

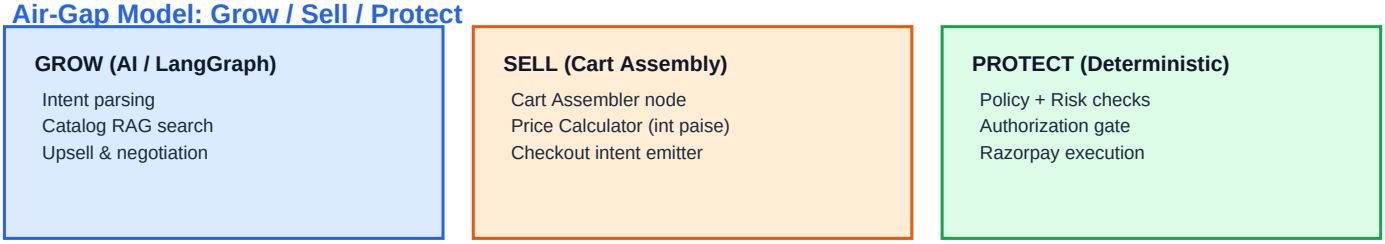
Version 1.0.0 | August 2026

1. Engineering Philosophy

As a rule, AI models are non-deterministic and therefore implicitly untrusted with financial execution. KeenPay is designed with a strict Air-Gap philosophy: the AI engine handles the Grow (upselling, negotiation, discovery) and Sell (cart assembly) logic, but all financial API calls to Razorpay are firmly locked behind a deterministic Protect layer.

We prioritize a pragmatic, monolithic-first API using FastAPI, PostgreSQL, and Redis. It is simple enough to deploy in minutes, but robust enough to handle enterprise concurrency and strict audit compliance.

Air-Gap Model: Grow / Sell / Protect

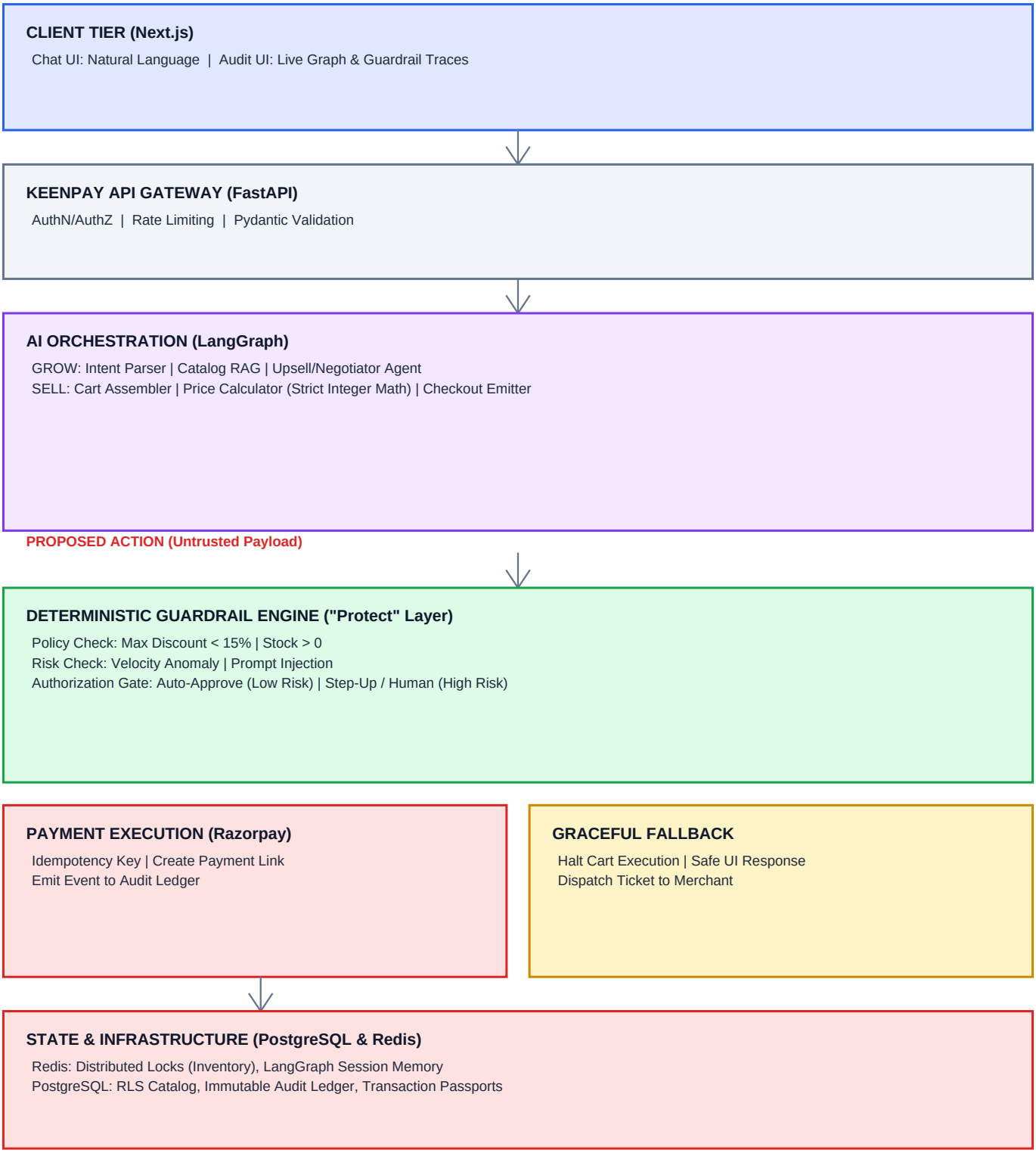


Trust Boundaries

- UNTRUSTED: User input, LLM output, inbound webhooks (validated on receipt)
- SEMI-TRUSTED: LangGraph orchestration (proposes actions, never pays)
- TRUSTED: Guardrail Engine, integer math, Razorpay client (gated side effects)

2. System Architecture (Grow & Sell)

This architecture separates conversational intelligence from the financial control plane. The client tier uses HTTPS/WebSockets for chat and SSE for live audit traces.



3. Workflow Execution Steps

The lifecycle of a KeenPay interaction is strictly linear. An AI cannot skip a step.

Phase 1: GROW (Discovery & Revenue Optimization)

1. User Intent

Buyer asks: "I need a mechanical keyboard, but only if I can get a discount on two."

2. Context Retrieval

LangGraph queries PostgreSQL/pgvector catalog for mechanical keyboards and retrieves buyer LTV.

3. Agentic Upsell

Agent replies: "I can offer the Keychron K2. If you buy two, I can apply a 10% bundle discount."

4. User Agreement

User agrees. Cart Assembler constructs JSON: quantity=2, discount=10% (proposed, not final).

Phase 2: PROTECT (The Guardrail Interception)

Before any API call to Razorpay, the payload is intercepted by the Python Guardrail Engine.

1. Inventory Lock

Redis attempts to claim stock by 2. If it fails, checkout halts immediately.

2. Policy Assertion

Python asserts proposed_discount <= merchant.max_discount_limit. AI-hallucinated 50% is clamped to 15% or blocked.

3. Price Re-computation

Final total calculated in integer paise by backend, NOT the LLM, to prevent tampering.

4. Risk Scoring

Unusually large transactions trigger SEND TO HUMAN workflow, halting automated checkout.

Phase 3: SELL (Razorpay Execution & Settlement)

1. Idempotent Order Creation

Backend generates unique cart hash as Idempotency-Key. Calls Razorpay POST /v1/payment_links.

2. Link Delivery

Payment Link pushed via WebSocket to Next.js chat interface.

3. Webhook Reconciliation

User pays. Razorpay fires payment_link.paid. KeenPay verifies HMAC webhook signature.

4. Audit Commit

Final state written to audit_ledger. Transaction Passport links Razorpay Order ID to LLM node and policy version.

Linear Flow Enforcement

```
GROW -> PROPOSE -> PROTECT -> [APPROVED] -> SELL -> SETTLE  |  [REJECTED] -> FALLBACK
```

4. Hardcoded Guardrails (Security Matrix)

To guarantee system integrity, these guardrails are enforced at the API level. No LLM prompt can override them.

Guardrail Vector	Threat	KeenPay Deterministic Solution
Data Isolation	Cross-merchant data leaks	Row-Level Security (RLS) in PostgreSQL. Merchant A cannot query Merchant B catalog or orders.
Double Spend	Network timeout causes payment retr	Strict Idempotency + Never-Retry-Unknown protocol. Razorpay timeout marks state UNKNOWN; background worker polls.
Margin Erosion	Prompt injection forces Rs.1 offer	Absolute Price Floor Policy. Backend recalculates (Base Price * Qty) - Max Allowed Discount.
Concurrency	Two agents sell last item	Redis Distributed Locks claim inventory before Payment Link generation; release on timeout.
Untraceability	Cannot debug bad transaction	Transaction Passport: immutable log linking Razorpay Order ID to LLM prompt and policy version.
Prompt Injection	Bypass rules via chat	Deterministic regex + anomaly scorer; security_block halts money actions.
LLM Arithme		
Webhook Ta		

Authorization Gate Outcomes

AUTO-APPROVE	Low risk: discount within policy, stock available, anomaly score < 0.5
CLAMP	Discount exceeds cap: force to merchant.max_discount_limit
REJECT	Margin violation or invalid SKU: halt, explain to user
STEP-UP / HUMAN	High risk or max negotiation rounds: escalation ticket to merchant

5. Transaction Passport & Audit Ledger

Every finalized cart generates an immutable Transaction Passport stored in PostgreSQL audit_logs. This proves exactly which AI node proposed the discount and which policy version authorized it.

Passport Fields

`passport_id`	UUID v4 primary reference
`session_id`	LangGraph thread / negotiation session
`razorpay_order_id`	Razorpay Payment Link or Order ID
`offer_version`	Monotonic cart proposal version
`decision_id`	Guardrail evaluation reference
`policy_version`	e.g. 2026.08.1
`grow_node_trace`	JSONB: intent, catalog hits, negotiation rationale
`protect_node_trace`	JSONB: per-rule pass/fail/clamp results
`sell_node_trace`	JSONB: idempotency_key, link_id, webhook events
`final_amount_paise`	Deterministic integer total
`created_at`	UTC timestamp (append-only)

Observability Channels

- WebSocket trace:{session_id} - real-time graph node + guardrail events to Audit UI
- SSE /ws/v1/session - Server-Sent Events for live state transitions
- PostgreSQL audit_logs - durable append-only ledger (UPDATE/DELETE blocked by trigger)
- Redis pub/sub - fan-out trace events to connected clients

Deployment Stack

Frontend	Next.js 14 - Chat UI + Audit UI (split pane)
API	FastAPI - monolithic gateway + LangGraph runtime
Orchestration	LangGraph - KeenPayStateGraph with interrupt before payment
Database	PostgreSQL 15 - RLS, pgvector catalog, audit ledger
Cache	Redis 7 - locks, session memory, rate limits, pub/sub
Payments	Razorpay Payment Links API (test + production modes)